Lambda Expressions

Functional interfaces, lambdas, method references, and comparators

Some methods need to be told *what to do*, not just what to work on: sorting needs to know the order, and removing needs to know which elements to remove. A *lambda expression* is a way to write that behavior in one short expression and pass it to a method. This part covers the interfaces lambdas work with, how to write lambdas and method references, the one rule about variables they use, and the collection methods that take them. The next part, on streams, builds on all of it. For reading, see Core Java, Volume I, Section 6.2. In class, the Speedrun Leaderboard example uses lambdas from Checkpoint 5 on.

A note about these notes. I wrote these notes with help from AI, from the examples and explanations I use in class. The AI drafted much of the text and code, and I review and revise them. Every example was compiled and run, and every output and error message shown is real; long stack traces are shortened. If something looks wrong, does not match what you see in Eclipse, or contradicts the book, tell me. The textbook is the tiebreaker, and the compiler beats us both. These notes do not replace the assigned chapters.

1. The Examples

The examples use one small record and a playlist of seven songs. The songs and artists are invented.

```
public record Song(String title, String artist, int seconds) { }
```

Table 1: The playlist used in the examples, in its original order.

Title	Artist	Seconds
Midnight Drive	Neon Harbor	254
Paper Moons	The Lanterns	198
Slow Burn	Neon Harbor	312
Northern Lights	Ada Vale	287
Echo Park	The Lanterns	221
Glass Garden	Ada Vale	345
Static	Neon Harbor	176

`Playlist.songs()` returns these songs as a `List<Song>` that cannot be changed.

2. Passing Behavior

`List` has a `sort` method that takes a `Comparator`: an object whose `compare` method decides which of two elements comes first. Before lambdas, the usual way to create one was an *anonymous class*, a class with no name written right where it is used:

```
List<Song> songs = new ArrayList<>(Playlist.songs());
```

```
songs.sort(new Comparator<Song>() {
    @Override
    public int compare(Song a, Song b) {
        return Integer.compare(a.seconds(), b.seconds());
    }
});
System.out.println(songs.get(0).title() + " is shortest");
```

That is six lines to say one thing: compare by length. A lambda says it in one, here reversed to put the longest song first:

```
songs.sort((a, b) -> Integer.compare(b.seconds(), a.seconds()));
System.out.println(songs.get(0).title() + " is longest");
```

```
Static is shortest
Glass Garden is longest
```

The lambda `(a, b) -> Integer.compare(b.seconds(), a.seconds())` is the body of the `compare` method and nothing else. Java fills in the rest, because `sort` expects a `Comparator<Song>`.

3. Functional Interfaces

A lambda can be used wherever Java expects an object of a *functional interface*: an interface with exactly one abstract method. `Comparator` is one. The package `java.util.function` provides the others you will use most, listed in Table 2.

Table 2: Common functional interfaces.

Interface	Method	Takes and returns
`Predicate<T>`	`boolean test(T t)`	Takes a `T` and answers true or false
`Function<T, R>`	`R apply(T t)`	Takes a `T` and returns an `R`
`Consumer<T>`	`void accept(T t)`	Takes a `T` and returns nothing
`Supplier<T>`	`T get()`	Takes nothing and returns a `T`
`BinaryOperator<T>`	`T apply(T a, T b)`	Takes two `T`s and returns a `T`
`Comparator<T>`	`int compare(T a, T b)`	Takes two `T`s and returns their order

```
Song song = new Song("Glass Garden", "Ada Vale", 345);

Predicate<Song> isLong = s -> s.seconds() > 300;
Function<Song, String> label = s -> s.title() + " (" + s.artist() + ")";
Consumer<String> show = text -> System.out.println("> " + text);
Supplier<List<Song>> emptyQueue = () -> new ArrayList<>();
BinaryOperator<Integer> add = (a, b) -> a + b;

System.out.println(isLong.test(song));
```

```
System.out.println(label.apply(song));
show.accept("now playing");
System.out.println(emptyQueue.get().size());
System.out.println(add.apply(254, 198));
```

```
true
Glass Garden (Ada Vale)
> now playing
0
452
```

A lambda stored in a variable is an object like any other: it can be passed to a method, returned, or kept in a field, and it runs only when its method is called. You can write functional interfaces of your own, too; marking one with `@FunctionalInterface` makes the compiler check that it has exactly one abstract method.

4. Lambda Syntax

A lambda has parameters, an arrow, and a body. Table 3 shows the forms.

Table 3: The forms of a lambda expression.

Lambda	Form
`s -> s.seconds() > 300`	One parameter: the parentheses are optional
`(a, b) -> a + b`	Two or more parameters: parentheses required
`() -> new ArrayList<>()`	No parameters: empty parentheses
`(Song s) -> s.title()`	Parameter types written out, which is optional
`s -> { ...; return x; }`	A block body, with `return` if it returns a value

An expression body needs no `return`: its value is the result. A body that needs several statements goes in braces, like a method body:

```
Function<Song, String> length = s -> {
    int minutes = s.seconds() / 60;
    int seconds = s.seconds() % 60;
    return String.format("%d:%02d", minutes, seconds);
};
System.out.println(length.apply(new Song("Slow Burn", "Neon Harbor", 312)));
```

```
5:12
```

If a lambda grows beyond a few lines, write a named method instead, and pass a method reference to it.

5. Method References

When a lambda does nothing but call one method, a *method reference* says the same thing more briefly, with two colons. There are four kinds, shown in Table 4.

Table 4: The four kinds of method reference.

Kind	Reference	The same lambda
A static method	`Integer::parseInt`	`s -> Integer.parseInt(s)`
A method of one object	`System.out::println`	`s -> System.out.println(s)`
A method of the argument	`String::length`	`s -> s.length()`
A constructor	`ArrayList::new`	`() -> new ArrayList<>()`

```
Function<String, Integer> parse = Integer::parseInt;       // a static method
Consumer<String> print = System.out::println;              // a method of one object
Function<String, Integer> length = String::length;         // a method of the argument
Supplier<List<String>> newList = ArrayList::new;           // a constructor

System.out.println(parse.apply("312") + 1);
print.accept("Echo Park");
System.out.println(length.apply("Echo Park"));
```

```
313
Echo Park
9
```

The third kind is the one used most with records: `Song::title` means `s -> s.title()`.

6. Variables Used in a Lambda

A lambda can use local variables from the method around it, as long as they are *effectively final*: never changed after they are given a value.

```
List<Song> songs = Playlist.songs();
int limit = 300;
songs.forEach(s -> {
    if (s.seconds() > limit) {
        System.out.println(s.title());
    }
});
```

```
Slow Burn
Glass Garden
```

Changing such a variable inside a lambda does not compile:

```
int count = 0;
```

```
songs.forEach(s -> count++);
```

```
error: local variables referenced from a lambda expression must be final or effectively final
```

A lambda is an object that may run later, after the method that created it has returned and its local variables are gone, so Java lets it use only a copy of a value that cannot change. To count, use a plain loop, or a stream's `count()`, which the streams notes cover.

7. Comparators

`Comparator` has methods that build comparators from smaller pieces, so you rarely write `compare` yourself:

```
songs.sort(Comparator.comparing(Song::artist).thenComparing(Song::title));
for (Song s : songs) {
    System.out.println(s.artist() + ": " + s.title());
}

songs.sort(Comparator.comparingInt(Song::seconds).reversed());
System.out.println("Longest first: " + songs.get(0).title() + ", "
        + songs.get(1).title() + ", " + songs.get(2).title());
```

```
Ada Vale: Glass Garden
Ada Vale: Northern Lights
Neon Harbor: Midnight Drive
Neon Harbor: Slow Burn
Neon Harbor: Static
The Lanterns: Echo Park
The Lanterns: Paper Moons
Longest first: Glass Garden, Slow Burn, Northern Lights
```

- `comparing(f)` orders by the value `f` returns, which must be comparable. `comparingInt` does the same for an `int`, without boxing.
- `thenComparing(f)` adds a tie-breaker, used only when the first comparison is a tie.
- `reversed()` flips the whole order.
- `Comparator.naturalOrder()` and `Comparator.reverseOrder()` use the elements' own `compareTo`, forward or backward.

One trap: `comparing` followed by `reversed` needs a method reference. With a plain lambda, Java cannot tell what type its parameter is:

```
Comparator<Song> longestFirst = Comparator.comparing(s -> s.seconds()).reversed();
```

```
error: cannot find symbol
  symbol:   method seconds()
```

Write `Song::seconds`, or give the parameter its type, `(Song s) -> s.seconds()`.

8. Lambdas with Collections

Several collection methods take a lambda, and they replace whole loops:

```
List<Song> songs = new ArrayList<>(Playlist.songs());
songs.removeIf(s -> s.seconds() < 200);
songs.forEach(s -> System.out.println(s.title()));

List<String> titles = new ArrayList<>(List.of("static", "echo park"));
titles.replaceAll(String::toUpperCase);
System.out.println(titles);
```

```
Midnight Drive
Slow Burn
Northern Lights
Echo Park
Glass Garden
[STATIC, ECHO PARK]
```

`removeIf` removes every element the predicate accepts, safely, which replaces the iterator loop from the collections notes. `forEach` runs a consumer on each element, and `replaceAll` replaces each element with what the function returns.

Maps have their own. `merge` counts in one line: if the key is new, it stores the given value; if not, it combines the old and new values with the function. `computeIfAbsent` groups in one line: it returns the key's value, creating it first if the key is new.

```
Map<String, Integer> plays = new TreeMap<>();
for (String title : List.of("Static", "Echo Park", "Static")) {
    plays.merge(title, 1, Integer::sum);
}
plays.forEach((title, count) -> System.out.println(title + ": " + count));

Map<String, List<String>> byArtist = new TreeMap<>();
for (Song s : Playlist.songs()) {
    byArtist.computeIfAbsent(s.artist(), artist -> new ArrayList<>()).add(s.title());
}
System.out.println(byArtist);
```

```
Echo Park: 1
Static: 2
{Ada Vale=[Northern Lights, Glass Garden], Neon Harbor=[Midnight Drive, Slow Burn, Static],
The Lanterns=[Paper Moons, Echo Park]}
```

The map's `forEach` takes a lambda with two parameters, the key and the value. The last output is one line, broken here to fit the page.

9. Summary

Table 5: Lambda expressions in brief.

Idea	Key points
Lambda	A method body with no name: parameters, an arrow, and an expression or a block.
Functional interface	An interface with one abstract method. A lambda becomes an object of it.
Method reference	`Type::method` when the lambda only calls one method. Four kinds.
Captured variables	Must be effectively final. A lambda cannot change a local variable.
Comparators	`comparing`, `thenComparing`, `reversed`; use method references with `reversed`.
Collection methods	`removeIf`, `forEach`, `replaceAll`, `sort`; on maps, `forEach`, `merge`, `computeIfAbsent`.

10. Try It

1. Rewrite the anonymous `Comparator` in Section 2 as a method reference chain with `Comparator.comparingInt`.
2. Write a `Predicate<Song>` that accepts songs by Neon Harbor, and use it with `removeIf` on a copy of the playlist. Which songs are left?
3. Sort the playlist by title length, shortest first, breaking ties alphabetically.
4. Use `merge` to build a map from each artist to the total seconds of that artist's songs.